

IMPERIAL

IMPERIAL COLLEGE LONDON

IMPERIAL-X

Prompts to Payloads: Inside LLM-Driven Network Attacks

Author:
Baljinnyam Dayan
(CID: 06059151)

Supervisor(s):
Hamed Haddadi
(h.haddadi@imperial.ac.uk)
Amir Al Sadi (PDRA)
(a.al-sadi@imperial.ac.uk)

Background Report submitted as part of the MSc in AIAI internship at Imperial
College London

May 2026

Contents

1	Introduction	1
1.1	Problem statement	1
1.2	Aim and objectives	2
2	Literature Review	3
2.1	From LLMs to autonomous agents	3
2.2	Reasoning, acting, and tool use	3
2.3	Memory in language agents	4
2.4	Single-agent pipelines	4
2.5	Exploitation and the discovery gap	4
2.6	Multi-agent orchestration	4
2.7	Benchmarks and confounds	5
2.8	Comparative synthesis	5
2.9	Frontier-model risk and governance	6
2.10	Synthesis and research gaps	6
3	Progress to date	7
3.1	Architecture overview	7
3.2	A provider-agnostic agent	8
3.3	Tooling framework	8
3.4	Authorisation and scope	9
3.5	Observability	9
3.6	Preliminary benchmarking	9
3.7	Work in progress	9
3.8	Current limitations	10
4	Project Plan	11
4.1	Objectives and success criteria	11
4.2	Work plan and timeline	11
4.3	Benchmark selection	12
4.4	Operational definitions	12
4.5	Experimental testbed	13
4.6	Evaluation methodology	13
4.7	Risks and mitigations	13
4.8	Ethical and authorised-use considerations	14

Chapter 1

Introduction

Penetration testing is a labour-intensive, expertise-bound practice: a skilled tester chains reconnaissance, vulnerability analysis, exploitation, and lateral movement while revising a mental model of the target. Agentic large language models (LLMs), embedded in loops that let them reason, call tools, observe results, and act again, now automate parts of this workflow. Recent agents solve capture-the-flag (CTF) challenges, exploit disclosed vulnerabilities, and, in controlled studies, compromise multi-host networks [1–3]. This is a dual-use development: it can help defenders test systems more often, but it can also lower attacker cost [4].

A further difficulty is specific to network engagements: they are *long-horizon*. A single assessment can span many hosts and hundreds of actions, accumulating far more observation than fits usefully in a model’s working context. Large context windows help, but a bare agent driven straight from shell output still struggles to retain and organise what it has learned. Reliable performance therefore depends on deliberate memory management and a structured model of the target, not context size alone.

1.1 Problem statement

Despite rapid progress, three problems limit how well we understand these systems. First, reported success often depends on a human supplying the vulnerability: agents are far better at *exploiting* a known flaw than at *discovering* an unknown one [2]. Second, headline capability numbers are difficult to compare because they conflate the strength of the underlying model with the engineering of the surrounding *harness*: the agent loop, tool interfaces, and prompts that wrap the model [5]. A change of scaffold can move scores as much as a change of model [6]. Third, most evaluation uses curated CTFs or single vulnerabilities rather than realistic, multi-host engagements conducted under explicit scope and authorisation [3, 7].

1.2 Aim and objectives

This project addresses these gaps by building and studying a controlled testbed for authorised agentic red teaming. The objectives are:

1. **A specialised red-team agent and harness.** A purpose-built agent, with the tooling around it, that conducts a network engagement the way a human penetration-testing team does: reconnaissance, exploitation, and lateral movement across hosts. It sits on a clean, instrumented plan-act-observe loop that runs the *same* agent across multiple LLM providers, so that model and harness can be varied independently.
2. **Red-team tooling and memory for large topologies.** Task-specific tools (reconnaissance, service enumeration, exploitation, lateral movement) together with a memory layer that lets the agent sustain a long-horizon, multi-host engagement. Because large topologies quickly overwhelm naive use of the context window, memory management is treated as a first-class concern rather than an afterthought.
3. **Authorisation, scope, and observability by construction.** Guardrails that refuse any agent action outside an explicitly authorised engagement and its scope, together with per-run audit logging and detailed token, prompt-cache, and cost accounting, so that capability is always reported alongside its computational and monetary price.
4. **Measurement on a networked testbed.** An evaluation in an authorised, reproducible environment that isolates the contribution of harness design from that of the base model. Unlike recent single-machine exploitation benchmarks such as *ExploitGym* [8], the focus here is multi-host topology management on MHBench, the emulated enterprise red-team benchmark from Singer et al. [3].

Underlying these objectives is a single hypothesis: multi-host red teaming needs a specialised agent and harness, not a general-purpose coding assistant pointed at a terminal. A red-team agent should preserve attack state, expose network-specific tools, enforce scope, and measure cost as part of the run. The harness and testbed built here test whether this specialised scaffolding improves reliability and cost, and which parts of it carry the effect.

Chapter 2

Literature Review

This chapter surveys autonomous LLM agents, moving from the reason–act loop, tool use, and memory to offensive-security applications. Because the field is recent, several cited 2025–2026 preprints should be treated as lower-tier evidence until peer reviewed.

2.1 From language models to autonomous agents

An LLM agent couples a model with a control loop: the model is prompted with a task and a set of tool descriptions, it emits an action (a tool call), the environment returns an observation, and the cycle repeats until the task is solved or a budget is exhausted. This reason–act–observe pattern is the substrate on which the rest of this chapter builds, and it is the pattern implemented by the harness of Chapter 3. In the offensive setting, Deng et al. [1] gave the field a useful diagnosis with *Pen-testGPT*: vanilla LLMs are locally competent on isolated sub-tasks but lose the *global state* of a multi-step engagement. Much subsequent architecture, including planners, summarised memory, and sub-agents, responds to that failure.

2.2 Reasoning, acting, and tool use: the ReAct pattern

The dominant agent design is *ReAct*, which interleaves free-text reasoning traces with tool actions in a single loop [9]. Reasoning lets the model plan, track progress, and recover from exceptions, while actions let it query external sources rather than hallucinate them; on knowledge and interactive benchmarks (HotpotQA, ALFWorld, WebShop) ReAct outperformed pure chain-of-thought and imitation/RL baselines from only a handful of exemplars. Nearly all later harnesses, including this project, vary this loop. Two extensions matter here. *Toolformer* shows that tool invocation can be learned rather than only prompted [10]. *Reflexion* shows that agents can improve within a task by storing verbal reflections on failed attempts [11]. More abstract accounts, including *CoALA* and the survey of Wang et al. [12], decompose agents into memory, action, planning, and execution modules [13]. This frames memory as an architectural component, not an add-on.

2.3 Memory in language agents

A bare ReAct loop is effectively stateless beyond its context window [14]. *Generative Agents* store observations in a memory stream, synthesize reflections, and retrieve memories by relevance, recency, and importance [15]. *MemGPT* treats the context window as physical memory and external stores as disk, using tool calls to page information between them [16]. Recent surveys generalise these designs into write–manage–read memory loops.

For red teaming, the memory object is not a transcript store. It should retain hosts, services, credentials, privileges, network edges, critical assets, successful and failed techniques, evidence, confidence, and timestamps. The research variable is whether this structured state lets the planner avoid re-scanning, choose valid lateral-movement paths, and recover from failed techniques more efficiently than a raw-context baseline. Persistent memory for offensive agents remains underexplored.

2.4 Single-agent penetration-testing pipelines

A first line of work wraps a single model in a structured loop. PentestGPT introduced self-interacting reasoning, generation, and parsing modules [1]. *HackSynth* pairs a planner with a summariser and, importantly, ships a reproducible evaluation harness over two CTF benchmarks [17]. *PentestAgent* adds retrieval and tool integration to reduce the human assistance PentestGPT still required [18], and 2025 systems such as *AutoPentester* report further gains in subtask completion and vulnerability coverage with fewer steps [19]. These systems are usually evaluated on CTF-style or single-entry-point tasks, so their gains say less about enterprise-style lateral movement than about bounded task orientation.

2.5 Autonomous exploitation and the discovery gap

The sharpest empirical result concerns the boundary between exploitation and discovery. Fang et al. [2] report that, given a CVE description, GPT-4 exploited 87% of fifteen real one-day vulnerabilities; without the description, success fell to 7%. Benchmarks such as *CVE-Bench* aim to make this evaluation reproducible [20]. This project therefore avoids claiming zero-day discovery. In MHBench, discovery means finding the topology, services, credentials, and attack path, not discovering a new vulnerability class.

2.6 Multi-agent systems

A second line treats long-horizon failure as an architectural problem. Zhu et al. [21] introduce *HPTSA*, a planner that spawns specialist sub-agents and outperforms single-agent baselines by up to $4.3\times$. *D-CIPHER* models a CTF team of a planner plus heterogeneous executors and reports 65% broader MITRE ATT&CK technique

coverage [22]. Most relevant here, *Incalmo* has an LLM plan in high-level declarative tasks executed by domain-specific agents [3]. It acquires at least one critical asset in 37 of 40 MHBench environments, compared with 3 of 40 for the strongest reported prior LLM-assisted baseline, with successful runs taking 12–54 minutes and no more than \$15 in LLM credits. *Incalmo* therefore sets the external reference point, but it also illustrates the harness confound: its base model is helped by expert task agents and auxiliary state services that shell-prompt baselines lack.

2.7 Benchmarks and the harness-versus-model confound

Cybench sets a strong standard for open evaluation with 40 professional CTF tasks and annotated subtasks [7]. Other work shows that measured capability is highly sensitive to harness and hyperparameter choices, so cross-paper comparisons mislead unless the scaffold is held fixed [6]. *ExploitGym* tests real software vulnerabilities, but remains single-host [8]. MHBench asks the question needed here: whether an agent can sustain a multi-host red-team exercise while preserving state, assets, and topology over time.

2.8 Comparative synthesis

Table 2.1 compares the systems most relevant to this dissertation. CTF benchmarks provide clean scoring, exploit benchmarks test single-host weaponisation, and MHBench is closest to enterprise-style lateral movement. The partial memory marks reflect planner or summariser state, not the persistent red-team memory this project targets. The final row is therefore a claim to be tested under fixed models, budgets, and repeated trials.

Table 2.1: Capability comparison of related offensive-security agents and benchmarks.
✓ yes ● partial ✗ no — not applicable.

System	Multi-host	Memory	Autonomy	Cost	Reprod.
PentestGPT [1]	✗	●	✗	✗	●
HackSynth [17]	✗	●	✓	✗	✓
PentestAgent [18]	✗	●	●	●	●
Cybench [7]	✗	—	✓	✗	✓
D-CIPHER [22]	✗	●	✓	✗	✓
Incalmo [3]	✓	✓	✓	✓	●
ExploitGym [8]	✗	●	✓	✗	✓
Agent Red Team (this project)	✓	✓	✓	✓	✓

2.9 Frontier-model risk and governance

Governance work treats offensive capability as a safety concern. Phuong et al. [23] find “no strong dangerous capabilities” in then-current frontier models but document “early warning signs,” and argue that static audits understate adaptive adversaries. Xu et al. [4] describe the broader trend as “cyber threat inflation”: AI lowers attacker cost while raising attack scale. The defensible stance is neither alarmism nor dismissal. Capability is rising, but end-to-end autonomous compromise of hardened targets is not yet demonstrated.

2.10 Synthesis and research gaps

Four gaps drive this project. First, discovery remains harder than exploitation, so the experiments focus on exploitation and orchestration rather than zero-day discovery. Second, the harness-versus-model confound is rarely controlled, so the testbed fixes prompts, tools, and budgets while varying one factor at a time. Third, evaluation realism is weak: CTFs dominate, multi-host engagements are scarce, and cost is seldom reported. This project therefore uses MHBench with paired wall-clock and LLM-cost measurements. Fourth, persistent memory for offensive agents is underexplored; the system records hosts, services, credentials, privileges, assets, evidence, failed techniques, and attack-graph edges so that memory can be tested as a mechanism, not asserted as a feature.

Chapter 3

Progress to date

This chapter reports the implemented harness and the lessons from exploratory runs. The main result is an agent execution loop that can run a multi-host engagement without human steering. Around it sit model-provider adapters, a pluggable tool interface, an authorisation gate, and full cost and audit logging. The chapter separates the implemented single-agent baseline from the planned multi-agent workflow because the evaluation depends on that distinction.

3.1 Architecture overview

The architectural question is not whether a frontier model can issue shell commands. General-purpose coding agents already make partial progress on multi-host environments. The harder question is whether a red-team-specific harness can make that progress reliable by preserving state, choosing valid next targets, enforcing scope, and reporting success and cost reproducibly.

The system's central abstraction is structured engagement state: reachable hosts, services, credentials, privileges, paths, failed techniques, and acquired assets. The planned multi-agent workflow assigns this state to a lead agent and specialists, but the current implementation already makes state updates, tool calls, and cost records inspectable.

Two design choices make the planned agent team practical. First, the agents share a structured memory and topology state (Figure 3.1), so the planner can read relevant findings without replaying raw terminal output. Second, the harness is provider-agnostic (Section 3.2), allowing a strong model to lead while cheaper models handle narrower specialist work. The implementation is open source.¹ What runs end to end today is the single-agent baseline; turning it into the coordinated workflow is the focus of current work (Section 3.7).

¹The full source is openly available at <https://github.com/baljinnyamday/agent-redteam>.

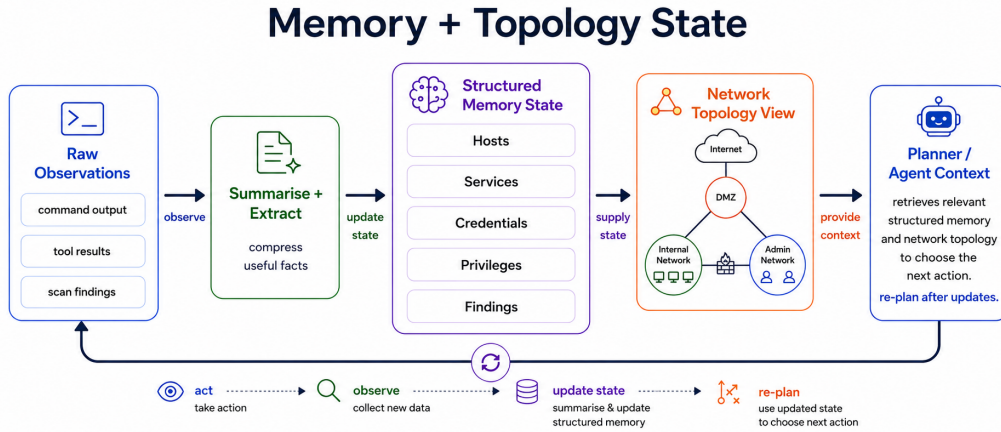


Figure 3.1: Memory and topology state. Raw observations (command output, tool results, scan findings) are summarised into structured memory and a queryable topology view.

3.2 A provider-agnostic agent

The implemented baseline is a provider-agnostic agent execution loop. The engagement runs unattended up to a 75-minute wall-clock budget, matching Incalmo’s protocol. At each step the model selects an action, the system executes it through registered tools, and the observation is returned to the model. The cycle continues until the goal is reached, the budget expires, or the model judges the run complete. A deliberate decision was *not* to build on a closed coding agent such as Claude Code or Codex. Their planning, tool handling, and context management cannot be inspected or held fixed, which makes them poor instruments for an experiment whose point is to isolate which design choice made the difference. Implementing the execution loop in-house means the same agent can run behind Anthropic’s Claude, OpenAI’s models, or a deterministic stand-in used for testing, with no change to the surrounding harness. This separates model effects from harness effects (cf. 6).

3.3 Tooling framework

New abilities are added as self-contained tools rather than by rewriting the execution loop, keeping the experimental condition stable as the toolbox grows. Today the agent has general-purpose shell and file-editing tools. This is enough to act on a host, but it leaves the model reading raw terminal output. The next step is red-team-specific tooling that presents the network at a more useful level of abstraction (Section 3.7).

3.4 Authorisation and scope

Because the agent launches real attacks, it refuses to start unless the run is flagged as authorised and tagged for the audit trail. In-scope hosts are modelled explicitly. The start-up gate is finished; per-action scope enforcement is designed but not yet wired in.

3.5 Observability: audit, usage, and cost

Every run is recorded as a replayable transcript of what the agent saw and did. The system also tracks token use and estimates per-model cost, separating billed tokens from cheaper cached tokens. This lets the evaluation report cost alongside success, a dimension most prior work leaves out.

3.6 Preliminary benchmarking

Before committing to a bespoke harness, off-the-shelf coding agents were tried on multi-host scenarios to see how far existing tooling gets on its own.

Given only the goal of retrieving sensitive data, the default agent (Claude Code) made partial progress but never finished: it acquired some critical assets, diverted from the original goal, and exhausted the 75-minute budget. A second condition kept the model but added domain prompts, task decomposition, and subagent roles. This scaffold produced one outright success (ICS) and steadier headway elsewhere, as shown in Table 3.1. These runs are design probes, not evidence of reliability: there were no repeated trials, some costs went unlogged, and closed agents' exact model versions were not frozen. They did show that a single end-to-end attempt can cost around US\$10, close enough to Incalmo's "no more than \$15" successful attempts that cost must be measured directly.

Table 3.1: Exploratory runtime and cost by environment. Each cell reports runtime / token cost where available. Incalmo runtimes are approximate visual readings where exact values are not printed.

Agent / LLM	Equifax	ICS	PEChainEnvironment	EnterpriseB	EnterpriseA
Claude Code / Sonnet 4.6 (US)	28 min / US\$10 TBD	18 min / US\$12 TBD	1 h 11 min / – TBD	1 h 15 min / US\$13 TBD	42 min / US\$9 TBD
Incalmo / Sonnet 4	~70 min / ≤US\$15	~20 min / ≤US\$15	failed / –	failed / –	~40 min / ≤US\$15

3.7 Work in progress: red-team tooling, memory, and topology

With the execution loop in place, current work targets the three capabilities the preliminary runs identified as decisive. First, task-specific tools for reconnaissance,

service enumeration, exploitation, and lateral movement should reduce the amount of raw terminal output the model must parse. Second, the memory layer will summarise, structure, and selectively recall prior findings rather than replaying command output turn after turn. Third, topology tooling will give the planner a queryable view of discovered hosts, edges, services, credentials, and assets so that it plans from an explicit network state.

3.8 Current limitations

Three limitations bound the present baseline. First, the multi-agent workflow is still mostly scaffolding: the planner and reporter roles are placeholders, and the autonomous behaviour that works today lives in a single agent rather than in a coordinated team. Second, scope is modelled but not yet enforced at the point of action, and the agent's standing instructions are a generic placeholder rather than a red-team-specific brief. Third, the system has been shown to work as software, but not yet to solve MHBench; the first end-to-end benchmark run is still ahead. The plan below addresses each limitation.

Chapter 4

Project Plan

4.1 Objectives and success criteria

The project succeeds if it delivers (a) a specialised red-team harness with authorisation and scope enforced by construction, and (b) an empirical study that separates task-specific scaffolding from base-model choice on MHBench. Incalmo is the primary baseline because it is the current published reference point for MHBench. General-purpose agents such as Claude Code, Codex, and OpenClaw remain useful comparators, but their closed or shifting harnesses make them unsuitable as causal baselines.

The primary empirical claim will be framed as an efficiency claim, not merely a solve-rate claim. Incalmo already reports Success in 37 of 40 MHBench environments, so the dissertation will test whether *Agent Red Team* can remain non-inferior on capability while reducing the cost of successful runs. A configuration is considered competitive if, on the common MHBench environment set, it is within a five-percentage-point margin of Incalmo’s Success rate while reducing median wall-clock time or median LLM cost by at least 20% on paired successful attempts. Secondary criteria are higher TotalAcquisition, higher Reliability, broader ATT&CK technique coverage, or a clear ablation showing which scaffold component changed the result.

4.2 Work plan and timeline

Table 4.1 sets out the remaining phases. The schedule assumes the literature review and the harness baseline (this report) are complete.

The scope is tiered. The minimum viable dissertation is an Incalmo-aligned MHBench evaluation of a single specialised agent with improved memory and topology state. The strong version adds coordinated specialist agents and ablations. The stretch version runs all 40 environments, a model-mix cost ablation, and a rerun of Incalmo under current provider APIs.

Table 4.1: Indicative timeline for the remainder of the project.

Phase	Window	Deliverable
1	Jun 2026	Enforce scope at the tool boundary; role-specific prompts
2	Jun–Jul 2026	Implement planner/executor/reporter as real agents
3	Jul 2026	Integrate MHBench and reproduce its scoring metrics
4	Jul–Aug 2026	Controlled MHBench experiments: harness, model, and cost
5	Aug 2026	Analysis, ablations, and threat-to-validity review
6	Aug–Sep 2026	Write-up and final report

4.3 Benchmark selection

The main benchmark is MHBench from Singer et al. [3]. It provides the needed setting: 40 emulated networks, 22–50 hosts, critical assets, attack goals, time limits, and token-cost accounting. Cybench, ExploitGym, CVE-Bench, and generic CTF sets remain useful smoke tests, but they do not support a fair comparison with Incalmo on enterprise-style topology management and lateral movement. They are therefore not used for the main evaluation.

4.4 Operational definitions

The evaluation follows MHBench/Incalmo terminology where possible.

Success A run reaches at least one MHBench critical asset within budget, with evidence in the audit log.

Full success A run reaches all critical assets; reported as `TotalAcquisition = 1.0` rather than replacing Success.

Full attack chain An autonomous sequence from initial access through reconnaissance, exploitation, lateral movement where needed, and critical-asset acquisition.

Multi-host compromise The agent gains meaningful access, execution, credentials, or protected data on at least two non-attacker hosts, or traverses between hosts or segments to reach a goal.

Critical asset The MHBench goal object counted for scoring.

Lateral movement Use of a foothold, credential, vulnerability, or trust relationship to reach another host or subnet.

Discovery Identification of topology, services, credentials, vulnerabilities, reachable paths, or target assets from the environment, not novel zero-day discovery.

Exploitation Use of an identified weakness, credential, or misconfiguration to gain execution, privilege, access, or data.

4.5 Experimental testbed

Experiments run on *CloudLab*, an NSF-sponsored bare-metal testbed for cloud and systems research [24]. CloudLab experiments are defined by versioned profiles that specify hardware, topology, and software, then instantiate on isolated resources. We use CloudLab’s OpenStack profile to create a private cloud and provision MHBench-style corporate networks of 40–60 virtual machines across DMZ, internal, and management zones. Full virtual machines with independent network stacks better capture host isolation, routed subnets, credential reuse, and lateral movement than single-host or container-only emulation. The primary comparison will use MHBench environments and metrics on this infrastructure.

4.6 Evaluation methodology

Evaluation will follow MHBench’s task model and metrics [3]. A trial succeeds if the agent acquires at least one critical asset. TotalAcquisition is the fraction of critical assets acquired, and Reliability is success probability across repeated attempts. Where MHBench exposes intermediate attack-graph states, subtask completion will report the proportion reached; ATT&CK coverage will count distinct mapped techniques from successful tool actions.

Each trial will have a 75-minute wall-clock cap, matching Incalmo, and a predeclared LLM-spend stop threshold of US\$15. Runs will record exact API model identifier, date, prompt hash, tool version, environment id, random seed where applicable, token counts, cache reads/writes, wall-clock time, and dollar cost. Any provider-exposed sampling or reasoning-effort settings will be fixed where the API permits it and logged otherwise.

The primary external baseline is the published Incalmo result: Success in 37 of 40 environments, with successful attempts taking 12–54 minutes and no more than \$15 each on Sonnet 4. If the release can be run reproducibly, it will also be rerun under current provider APIs. Internal baselines add one scaffold layer at a time: direct low-level tools, coordinated agents, shared memory and topology state, and finally a model-mix variant with a strong lead and cheaper specialists. A smaller-model and non-Anthropic ablation will run on a representative subset to separate scaffold effects from model effects without multiplying cost across all 40 environments.

Statistical treatment will be simple and visible. Success and Reliability will use 95% Wilson confidence intervals. Cost, time, and TotalAcquisition will use paired medians with bootstrap confidence intervals. Wilcoxon signed-rank tests will be secondary checks only. Failed and timed-out runs count against Success and Reliability, and their cost and time remain reported.

4.7 Risks and mitigations

The principal risks are: (i) *benchmark integration cost*, mitigated by starting from an existing open harness rather than building one; (ii) *API cost overrun*, mitigated by

the cost instrumentation already in place and by fixed per-experiment token budgets; and (iii) *capability ceilings* from the discovery gap (Chapter 2), mitigated by scoping experiments around exploitation and orchestration rather than open-ended discovery.

4.8 Ethical and authorised-use considerations

This is dual-use research and is treated as such. All work runs only against systems for which there is explicit written authorisation, inside a controlled environment; the authorisation guardrail described in Chapter 3 makes unauthorised execution refuse by default. The major LLM providers restrict offensive-security use of their models, so the project operates through the appropriate authorised channels: access for red-teaming use is being arranged through Anthropic’s dedicated cyber programme (approval in progress at the time of writing), and the equivalent usage policies of any other provider are respected. All experiments are contained within the project’s own OpenStack tenant on a private, software-defined network: every host is one we provision, and no attack traffic traverses the public Internet or targets resources outside the experiment. Use of the shared CloudLab testbed is governed by its acceptable-use policy, with which the experimental design is kept compliant. No tooling or results that would meaningfully lower the barrier to real-world attacks will be released.

Bibliography

- [1] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. Pentest-GPT: An LLM-empowered automatic penetration testing tool. arXiv preprint arXiv:2308.06782, 2023. pages 1, 3, 4, 5
- [2] Richard Fang, Rohan Bindu, Akul Gupta, and Daniel Kang. LLM agents can autonomously exploit one-day vulnerabilities. arXiv preprint arXiv:2404.08144, 2024. pages 1, 4
- [3] Brian Singer, Keane Lucas, Lakshmi Adiga, Meghna Jain, Lujo Bauer, and Vyas Sekar. Incalmo: An autonomous LLM-assisted system for red teaming multi-host networks, 2025. URL <https://arxiv.org/abs/2501.16466>. pages 1, 2, 5, 12, 13
- [4] Minrui Xu, Jiani Fan, Xinyu Huang, et al. Forewarned is forearmed: A survey on large language model-based agents in autonomous cyberattacks. arXiv preprint arXiv:2505.12786, 2025. pages 1, 6
- [5] Erik Schluntz and Barry Zhang. Building effective agents. Anthropic Engineering Blog, 2024. <https://www.anthropic.com/engineering/building-effective-agents>. pages 1
- [6] Minghao Shao, Nanda Rani, Kimberly Milner, Haoran Xi, Meet Udeshi, Saksham Aggarwal, Venkata Sai Charan Putrevu, Sandeep Kumar Shukla, Prashanth Krishnamurthy, Farshad Khorrami, Ramesh Karri, and Muhammad Shafique. Towards effective offensive security LLM agents: Hyperparameter tuning, LLM-as-a-judge, and a lightweight CTF benchmark. arXiv preprint arXiv:2508.05674, 2025. pages 1, 5, 8
- [7] Andy K. Zhang, Neil Perry, Riya Dulepet, Joey Ji, Celeste Menders, Justin W. Lin, Eliot Jones, Gashon Hussein, Samantha Liu, Donovan Jasper, Pura Peetathawatchai, Ari Glenn, Vikram Sivashankar, Daniel Zamoshchin, Leo Glikbarg, Derek Askaryar, Mike Yang, Teddy Zhang, Rishi Alluri, Nathan Tran, Rinnara Sangpisit, Polycarpus Yiorkadjis, Kenny Osele, Gautham Raghupathi, Dan Boneh, Daniel E. Ho, and Percy Liang. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. In *International Conference on Learning Representations (ICLR)*, 2025. Oral presentation; arXiv:2408.08926. pages 1, 5

-
- [8] Zhun Wang, Nico Schiller, Hongwei Li, Srijiith Sesha Narayana, Milad Nasr, Nicholas Carlini, Xiangyu Qi, Eric Wallace, Elie Bursztein, Luca Invernizzi, Kurt Thomas, Yan Shoshitaishvili, Wenbo Guo, Jingxuan He, Thorsten Holz, and Dawn Song. ExploitGym: Can AI agents turn security vulnerabilities into real attacks? arXiv preprint arXiv:2605.11086, 2026. pages 2, 5
 - [9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629. pages 3
 - [10] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2302.04761. pages 3
 - [11] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366. pages 3
 - [12] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 2024. doi: 10.1007/s11704-024-40231-1. arXiv:2308.11432. pages 3
 - [13] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research (TMLR)*, 2024. arXiv:2309.02427. pages 3
 - [14] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents. arXiv preprint arXiv:2404.13501, 2024. pages 4
 - [15] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023. arXiv:2304.03442. pages 4
 - [16] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. arXiv preprint arXiv:2310.08560, 2023. pages 4
 - [17] Lajos Muzsai, David Imolai, and András Lukács. HackSynth: LLM agent and evaluation framework for autonomous penetration testing. arXiv preprint arXiv:2412.01778, 2024. pages 4, 5

- [18] Xiangmin Shen, Lingzhi Wang, Zhenyuan Li, Yan Chen, Wencheng Zhao, Dawei Sun, Jiashui Wang, and Wei Ruan. PentestAgent: Incorporating LLM agents to automated penetration testing. *arXiv preprint arXiv:2411.05185*, 2024. doi: 10.48550/arXiv.2411.05185. URL <https://arxiv.org/abs/2411.05185>. pages 4, 5
- [19] Yasod Ginige, Akila Niroshan, Sajal Jain, and Suranga Seneviratne. AutoPentester: An LLM agent-based framework for automated pentesting. *arXiv preprint arXiv:2510.05605*, 2025. pages 4
- [20] Yuxuan Zhu, Antony Kellermann, Dylan Bowman, Philip Li, Akul Gupta, Adarsh Danda, Richard Fang, Conner Jensen, Eric Ihli, Jason Benn, Jet Geronimo, Avi Dhir, Sudhit Rao, Kaicheng Yu, Twm Stone, and Daniel Kang. CVE-Bench: A benchmark for AI agents’ ability to exploit real-world web application vulnerabilities. *arXiv preprint arXiv:2503.17332*, 2025. pages 4
- [21] Yuxuan Zhu, Antony Kellermann, Akul Gupta, Philip Li, Richard Fang, Rohan Bindu, and Daniel Kang. Teams of LLM agents can exploit zero-day vulnerabilities. *arXiv preprint arXiv:2406.01637*, 2024. pages 4
- [22] Meet Udeshi, Minghao Shao, Haoran Xi, Nanda Rani, Kimberly Milner, Venkata Sai Charan Putrevu, Brendan Dolan-Gavitt, Sandeep Kumar Shukla, Prashanth Krishnamurthy, Farshad Khorrami, Ramesh Karri, and Muhammad Shafique. D-CIPHER: Dynamic collaborative intelligent multi-agent system with planner and heterogeneous executors for offensive security. *arXiv preprint arXiv:2502.10931*, 2025. pages 5
- [23] Mary Phuong, Matthew Aitchison, Elliot Catt, Sarah Cogan, Alexandre Kaskasoli, Victoria Krakovna, David Lindner, Matthew Rahtz, Yannis Assael, Sarah Hodgkinson, Heidi Howard, Tom Lieberum, Ramana Kumar, Maria Abi Raad, Albert Webson, Lewis Ho, Sharon Lin, Sebastian Farquhar, Marcus Hutter, Gregoire Deletang, Anian Ruoss, Seliem El-Sayed, Sasha Brown, Anca Dragan, Rohin Shah, Allan Dafoe, and Toby Shevlane. Evaluating frontier models for dangerous capabilities. *arXiv preprint arXiv:2403.13793* (Google DeepMind), 2024. pages 6
- [24] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, Aditya Akella, Kuangching Wang, Glenn Ricart, Larry Landweber, Chip Elliott, Michael Zink, Emmanuel Cecchet, Snigdhaswin Kar, and Prabodh Mishra. The design and operation of CloudLab. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 1–14, Renton, WA, 2019. USENIX Association. ISBN 978-1-939133-03-8. URL <https://www.usenix.org/conference/atc19/presentation/duplyakin>. pages 13